

Salt Lake City AQI Forecasting

Basic Info:

Title: Salt Lake City AQI Forecasting.

Team Members:

- Michael Northrup, u0992000, michael.northrup@yahoo.com
- Grant (Rob) Olsen, u0684253, rolsenrob@gmail.com
- Dzevad Fitozovic, u0191522, dfitozovic@hotmail.com

Acknowledgements and References:

Code in this project was derived from and, in some cases, the technique is identical to code from <https://github.com/datascience-course> (<https://github.com/datascience-course>) and homework solutions. Ideas on how to approach a sample set which has only a few positive results, or highly skewed data, was obtained from: <https://florianhartl.com/thoughts-on-machine-learning-dealing-with-skewed-classes.html> (<https://florianhartl.com/thoughts-on-machine-learning-dealing-with-skewed-classes.html>). Code to do this was obtained from http://scikit-learn.org/stable/modules/cross_validation.html#cross-validation (http://scikit-learn.org/stable/modules/cross_validation.html#cross-validation).

Background and Motivation

We think that the quality of our air here in the valley is often a topic of discussion. However, the primary motivation came from one of our assignments. There we attempted to use the AQI to give us a moving average in order to be able to issue a warning to sensitive groups when an upwards trend was most likely occurring. We wanted to improve and expand on that, so in this project we want to see if we can predict the AQI for a few days in advance, creating a forecast for 5 days in advance. We originally planned to make an AQI long term forecast model, so that one would be able to see how many days of poor air quality we can expect to see as our valley population rises. We ended up dropping this aspect of the project due to the complexity of just the forecast problem. We do not have a background nor have we done any research in the area, but an AQI forecast seemed like an interesting problem.

Project Objectives

Our primary objective is to devise a model to obtain an AQI forecast much like we have a weather forecast. Short term AQI data combined with weather forecasts should be useful in creating a short term forecast model. A short term model can be useful in warning sensitive groups of upcoming poor air quality days. Early in the project, we determined the process of creating a model that beats extremely advanced model with access to thousands of parameters and manually gathered data would be almost impossible. In short, we would classify our model as the cheaper way to create this kind of air quality forecast, using less data that is more accessible. Along with that, we did not find another source for a forecast that goes beyond one day. This gives our project the unique aspect that we attempt to predict beyond just the next day's forecast to allow planning around air quality later in the week.

Deviations from Plans

We originally planed on doing a 1-5 day AQI forecast model and a 1-5 year population growth model. However, due to the challenges with the 1-5 day model we decided to focus our energy there. Additionally, we intended to get data for weather and AQI going back to 1990. However, weather data going back that far was too unreliable, since there was a lot of missing data. We therefore fetched monthly reports from <http://w2.weather.gov/climate/index.php?wfo=slc> (<http://w2.weather.gov/climate/index.php?wfo=slc>) then formatted the data outside of Python. Unfortunately, by this method we were only able to obtain 5 years of data going back to 11-01-2011. Another change we needed to make is our analysis threshold. We originally planned on trying to predict if a day will be greater than or less than 101. But because so few days actually matched this criteria even a dumb model that guessed that every day will be less them 101 was over 90% accurate. We therefore changed our threshold to 51, which attempts to predict if a day will be Green or not Green (i.e. Red). This proved a more realistic forecast for us to produce.

Data

We had to gather and clean a significant amount of data from several sources. For historic air quality data, the air quality data was collected from <https://www.epa.gov/outdoor-air-quality-data> (<https://www.epa.gov/outdoor-air-quality-data>). However, each year comes as a separate data-file, so it had to be cleaned and combined. Additionally, this data was not up to date. We therefore needed to obtain data and add it manually to the AQIOctNov datafile from the Air Now web site at https://airnow.gov/index.cfm?action=airnow.local_city&mapcenter=0&cityid=186 (https://airnow.gov/index.cfm?action=airnow.local_city&mapcenter=0&cityid=186). Reliable and complete historic weather data was difficult to obtain. We ended up retrieving and cleaning Preliminary Monthly Climate Data reports using OpenOffice from <http://w2.weather.gov/climate/index.php?wfo=slc> (<http://w2.weather.gov/climate/index.php?wfo=slc>). These reports only went back 5 years which was is why our datasets begin at 5 years from this November. Also, these reports have to be rerun to update our data to the most recent available numbers, manually adding them into our recent_climate_data file. Our short term weather forecast, is obtained from the API at <http://openweathermap.org/> (<http://openweathermap.org/>) which we use in our model to predict air quality. As we built the data frames we encountered issues with making sure column names agreed and how to handle data for "today", which needed to be forecasted. However, if the API was run after 5pm, the next day's data becomes available and "today's" data is lost. Some additional iddues we had where how to fill missing days and how to combine the dataframes from the different sources. Also, having to take the max of some columns, minimum of others, and mean of some was a hurdle we had to overcome. The date time also had to be tinkered with take make sure the data was in proper order.

```
In [1]: # imports
import pandas as pd
import requests
import json
import uuid
from IPython.display import display_javascript, display_html, display
# get needed regression and classification imports
import numpy as np
from sklearn import tree, metrics, svm, gaussian_process, neural_network
from scipy.stats import linregress
from sklearn.linear_model import SGDRegressor, BayesianRidge, Perceptron, LogisticRegression
from sklearn.kernel_ridge import KernelRidge
# from sklearn.neural_network import MLPRegressor
import statsmodels.formula.api as sm

# from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt
%pylab inline
%matplotlib inline
plt.rcParams['figure.figsize'] = (14,12)
plt.style.use('ggplot')
```

Populating the interactive namespace from numpy and matplotlib

Data Processing.

Most of the cleanup for the population data and 5 year weather data was completed in OpenOffice out of convenience. Also, we saw even more complexity in the data derived from the Weather API for the most recent weather forecasts. We expected that we should be able to give a fairly accurate short term AQI forecast, namely what type of air quality day one expects to have in the upcoming 5 days. Below is the querying and formatting of the API weather forecast data in preparation for predicting air quality. For our model to be appropriately predictive, it needed to have weather data and AQI data up to the day before this analysis is being performed. We found no good way to automate this, so this data must be updated manually. Additionally, once that data has been updated the API queries below must be run before 5 PM due to how the API returns data. If it is run after 5 pm predictions for today will not be available and the resulting prediction for the 5 day forecast will not be meaning full.

The most recent AQI data does not come from the same source as the data which is more than 2 months old. This was a limitation of the AQI historic data which we needed overcome to get a working model. We believe that obtaining the most recent AQI data from a different source was a good solution since the historic AQI data we obtained was averaged over the different POC sites. Therefore we expect there to be minimal measurement discrepancies between the two data sources. Similar logic was used to accept any possible discrepancies between the two sources for weather data.

```
In [2]: # Upload API data for the weather forecast and store it in a dictionary

# This block of code needs to be run before 5pm for a proper forecast. The forecast
# needs to include data for today,
# since today is a day that will be forecasted as well. After 5pm the API moves on
# to the next day.
# However, actual measured weather and AQI data for today will not be available until tomorrow.

response = requests.get("http://api.openweathermap.org/data/2.5/forecast?id=5780993
&APPID=e0c55e00bf021f6142de334823046e9e&units=imperial")
weather = response.content.decode("utf-8")
weatherDict = json.loads(weather)

In [3]: # print the raw dump file (commented out to save space)
# print(json.dumps(weatherDict, indent=2))
```

In [4]: *# PROCESSING OF WEATHER FORECAST DATA*

```
timeList = []
maxTempList = []
minTempList = []
pressureList = []
humidityList = []
tempList = []
wind_speedList = []
windDegList = []
precipitationList = []
rainList = []
rainListMM = []
snowList = []
snowListMM = []
```

Some weather forecast data was given in 3 hour increments this needed to be combined to obtain daily values.

```
for weatherEntry in weatherDict["list"]:
    timeList.append(weatherEntry["dt_txt"])
    mainWeather = weatherEntry["main"]
    maxTempList.append(mainWeather["temp_max"])
    minTempList.append(mainWeather["temp_min"])
    pressureList.append(mainWeather["pressure"])
    humidityList.append(mainWeather["humidity"])
    tempList.append(mainWeather["temp"])
    windWeather = weatherEntry["wind"]
    wind_speedList.append(windWeather["speed"])
    windDegList.append(windWeather["deg"])
    precip = 0

    # Obtain rain totals separately
    if "rain" in weatherEntry:
        if "3h" in weatherEntry["rain"]:
            rainList.append(1)
            rainListMM.append(weatherEntry["rain"]["3h"])
            precip = precip + weatherEntry["rain"]["3h"]
        else:
            rainList.append(0)
            rainListMM.append(0)
    else:
        rainList.append(0)
        rainListMM.append(0)

    # Obtain snow totals separately
    if "snow" in weatherEntry:
        if "3h" in weatherEntry["snow"]:
            snowList.append(1)
            snowListMM.append(weatherEntry["snow"]["3h"])
            precip = precip + weatherEntry["snow"]["3h"]
        else:
            snowList.append(0)
            snowListMM.append(0)
    else:
        snowList.append(0)
        snowListMM.append(0)
    precipitationList.append(precip)
```

```

In [5]: # setup of dataframe for 5 day forecast; Day 1 is today if the AIP code above was r
un before 5pm.
data = [('DateTime', timeList),
        ('MaxTemp', maxTempList),
        ('MinTemp', minTempList),
        ('Humidity', humidityList),
        ('Temperature', tempList),
        ('WindSpeed', wind_speedList),
        ('WindDeg', windDegList),
        ('Precipitation (mm)', precipitationList)
        ]

weatherForecast = pd.DataFrame.from_items(data)

#Cuts away the time stamps leaving only the date.
weatherForecast["DateTime"] = weatherForecast["DateTime"].apply(lambda x: str(x)[:10])

#Fills final dataframe with means for all columns
finalForecast = weatherForecast.groupby("DateTime").mean()

#Fills columns that need different grouping methods in to final dataframe
finalForecast["AVGtemp(F)"] = weatherForecast.groupby("DateTime").mean()["MinTemp"]
finalForecast["MINtemp(F)"] = weatherForecast.groupby("DateTime").min()["MinTemp"]
finalForecast["MAXtemp(F)"] = weatherForecast.groupby("DateTime").max()["MaxTemp"]
finalForecast["MaxWindSpeed(mph)"] = weatherForecast.groupby("DateTime").max()["WindSpeed"]
finalForecast["Avgwindspeed(mph)"] = weatherForecast.groupby("DateTime").mean()["WindSpeed"]

# Remove unused columns
del finalForecast["Humidity"]
del finalForecast["Temperature"]
del finalForecast["WindDeg"]
del finalForecast["MaxTemp"]
del finalForecast["MinTemp"]
del finalForecast["WindSpeed"]

# split the data for each day to be used recursively in the prediction models later
.
day1 = pd.DataFrame()
day2 = pd.DataFrame()
day3 = pd.DataFrame()
day4 = pd.DataFrame()
day5 = pd.DataFrame()
day1 = day1.append(finalForecast.iloc[0])
day2 = day2.append(finalForecast.iloc[1])
day3 = day3.append(finalForecast.iloc[2])
day4 = day4.append(finalForecast.iloc[3])
day5 = day5.append(finalForecast.iloc[4])
finalForecast

```

Out[5]:

	Precipitation (mm)	AVGtemp(F)	MINtemp(F)	MAXtemp(F)	MaxWindSpeed(mph)	Avgwindspee
DateTime						
2016-12-04	0.000000	28.6425	23.22	36.26	9.42	6.35250
2016-12-05	0.817187	32.0575	29.36	36.08	16.20	8.17750
2016-12-06	0.005625	23.5125	17.28	28.15	13.40	7.88000
2016-12-07	0.679688	21.9775	19.34	24.55	14.99	8.09000
2016-12-08	0.001250	15.3500	10.47	25.31	10.20	7.39625

In [6]: *# PROCESSING OF HISTORIC AQI DATA*

```
# Load AQI data from files and group by mean for each date so that we only have 1 value for each date
# which was averaged over the different Point of Collection sites.
aqi_data_2011 = pd.read_csv("2011.csv",parse_dates=True)
df2011 = pd.DataFrame(aqi_data_2011).groupby("Date").mean()

aqi_data_2012 = pd.read_csv("2012.csv",parse_dates=True)
df2012 = pd.DataFrame(aqi_data_2012).groupby("Date").mean()

aqi_data_2013 = pd.read_csv("2013.csv",parse_dates=True)
df2013 = pd.DataFrame(aqi_data_2013).groupby("Date").mean()

aqi_data_2014 = pd.read_csv("2014.csv",parse_dates=True)
df2014 = pd.DataFrame(aqi_data_2014).groupby("Date").mean()

aqi_data_2015 = pd.read_csv("2015.csv",parse_dates=True)
df2015 = pd.DataFrame(aqi_data_2015).groupby("Date").mean()

aqi_data_2016 = pd.read_csv("2016.csv",parse_dates=True)
df2016 = pd.DataFrame(aqi_data_2016).groupby("Date").mean()

# Data for most recent AQI was not downladable and therefore we aquired it from the AIR now web site manually.
# The datafile AQIOctNov.csv had to be updated manually from AIR Now site and does not match a particular POC.
# However, due to the fact that our daily data is itself an average value accross several POC sites, this seems
# like a good way to obtain most current AQI data.
aqi_data_present = pd.read_csv("AQIOctNov.csv",parse_dates=True)
dfpresent = pd.DataFrame(aqi_data_present).groupby("Date").mean()

# Combine data into a single dataframe
all_aqi_data = pd.DataFrame()
all_aqi_data = df2011.append(df2012)
all_aqi_data = all_aqi_data.append(df2013)
all_aqi_data = all_aqi_data.append(df2014)
all_aqi_data = all_aqi_data.append(df2015)
all_aqi_data = all_aqi_data.append(df2016)
all_aqi_data = all_aqi_data.append(dfpresent)

# Forward fill any empty values
all_aqi_data = all_aqi_data.fillna(method = 'ffill')

# Create a column for previous day AQI by shifting all enteries by 1.
    # Createing a Previous entry column this way would normally result in a missing value for the first entry
    # in the list. However, we needed to truncate the data to only values past 11-01-11 therefore we did not
    # need to back fill that value.
all_aqi_data["Previous_AQI_VALUE"] = all_aqi_data["DAILY_AQI_VALUE"].shift(1)

# Remove unused columns
del all_aqi_data["POC"]
del all_aqi_data["AQS_SITE_ID"]
del all_aqi_data["Daily Mean PM2.5 Concentration"]
del all_aqi_data["DAILY_OBS_COUNT"]
del all_aqi_data["PERCENT_COMPLETE"]
del all_aqi_data["AQS_PARAMETER_CODE"]
del all_aqi_data["CBSA_CODE"]
```

Out[6]:

	DAILY_AQI_VALUE	Previous_AQI_VALUE
count	1858.000000	1858.000000
mean	31.349543	31.328821
std	24.530530	24.518348
min	3.750000	3.750000
25%	17.000000	17.000000
50%	23.333333	23.333333
75%	34.333333	34.250000
max	159.333333	159.333333

Looking at the quintiles for the Daily AQI values, it appears like most of the time the AQI is well below 101. Therefore, a model that looks at predicting an AQI over 101 would probably be over 90% accurate if it only guessed that the AQI will be below 101 every day. Obviously that would be a useless model. So, we have modified our analysis to check if the AQI will be greater than or less than 51. This gives a more reasonable number of days where the model can fail. Originally we had moving averages columns but we removed these due to platform issues running on Mac, and because they ended up being mostly unnecessary to the power of our model.

```
In [7]: # Load weather data file
# This CSV file is updated manually for the previous day's data from the
weather_data_slc = pd.read_csv("recent_climate_data.csv")
# change Percipitation and Snow units from inches to mm to fit in with 5 day forecast data
weather_data_slc['Precipitation (mm)'] = weather_data_slc['Percipitation']*25.4
# Remove unused columns
del weather_data_slc['Percipitation']
del weather_data_slc['Snow']
# reindex by day of month.
weather_data_slc = weather_data_slc.set_index(["Date"])
weather_data_slc.describe()
```

Out[7]:

	MAXtemp(F)	MINtemp(F)	AVGtemp(F)	AvgWindSpeed(mph)	MaxWindSpeed(mph)	Precipita (mm)
count	1860.000000	1860.000000	1860.000000	1860.000000	1860.000000	1860.000000
mean	66.222581	44.636022	55.674731	7.765753	19.049462	0.981177
std	21.182242	17.256889	18.990927	3.342536	7.376432	2.974634
min	14.000000	-2.000000	7.000000	1.500000	5.000000	0.000000
25%	49.000000	32.000000	41.000000	5.500000	14.000000	0.000000
50%	66.000000	43.500000	55.000000	7.100000	17.000000	0.000000
75%	85.000000	59.000000	72.000000	9.500000	24.000000	0.254000
max	105.000000	81.000000	91.000000	22.800000	51.000000	31.750000

It appears like our precipitation data is highly skewed to the low end as indicated by the Min, 25% and 50% quantile values, which is as expected in Utah. The other data columns also appear to make sense. Due to problems with merging of forecast and historical data the snow data was removed and we kept only the precipitation data.

Exploratory Analysis.

We wanted to visualize how precipitation, temperature and wind speed affect the AQI by plotting the variables with the AQI values. Additionally, we created a scatter matrix of all variables used in predictions. One can definitely see trends for temperature. Precipitation and wind speed's effect on the AQI is less clear.

```
In [8]: # Combine AQI data with weather data

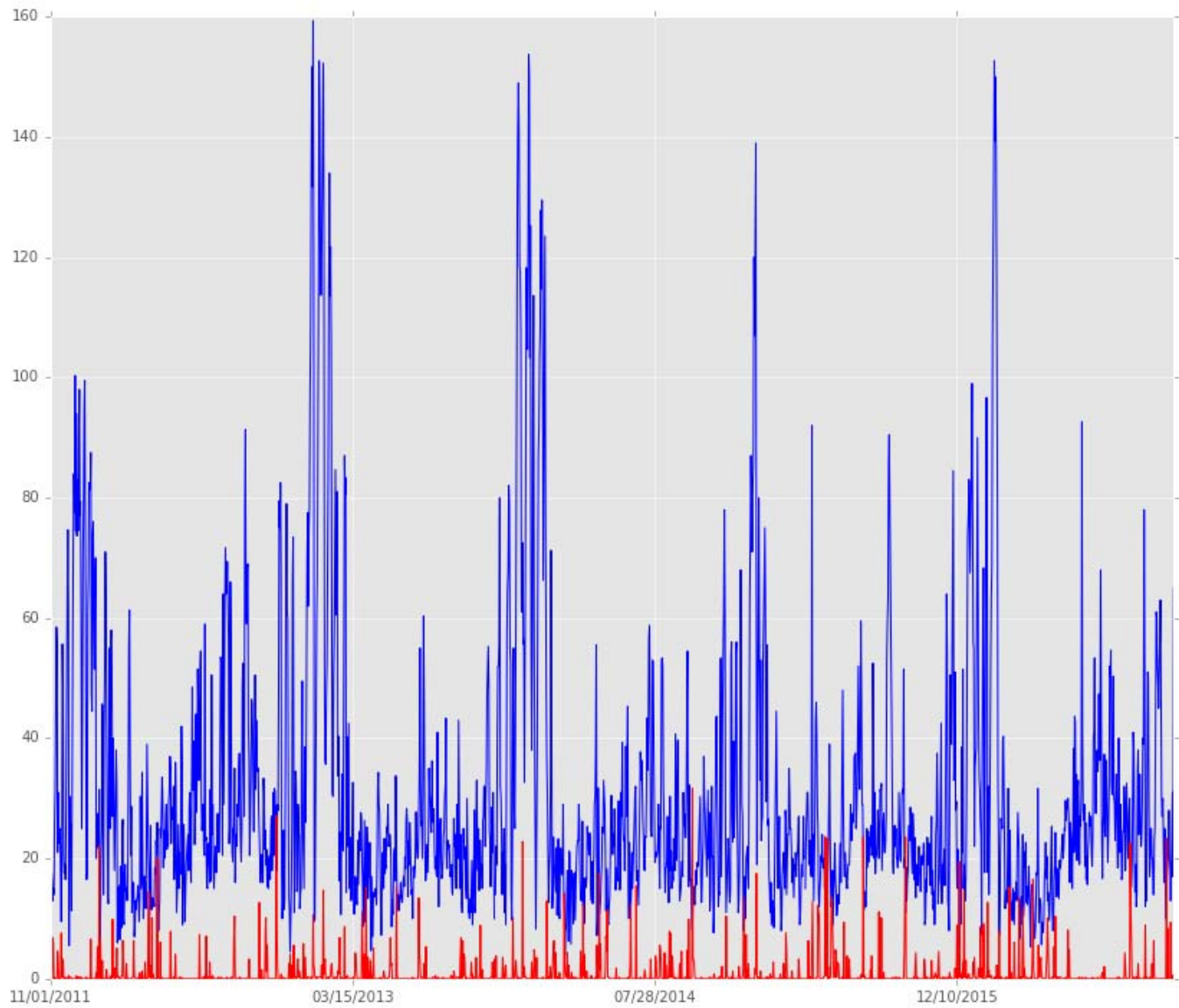
# only appending the Daily AQI value with weather data since a rolling average will
# not be possible for forecast data.
frames = [weather_data_slc, all_aqi_data]
all_historic_data = pd.concat(frames, axis = 1)

# create column of date, month, day, and year and resort data appropriately
# these will be removed later before analysis.
all_historic_data['Date'] = all_historic_data.index
all_historic_data['Month'] = all_historic_data['Date'].apply(lambda x: str(x)[:2])
all_historic_data['Day'] = all_historic_data['Date'].apply(lambda x: str(x)[3:5])
all_historic_data['Year'] = all_historic_data['Date'].apply(lambda x: str(x)[6:10])
all_historic_data['New Date'] = all_historic_data['Year']+all_historic_data['Month'
]+all_historic_data['Day']
all_historic_data['New Date'] = all_historic_data['New Date'].apply(int)
all_historic_data = all_historic_data.sort_values(by='New Date')

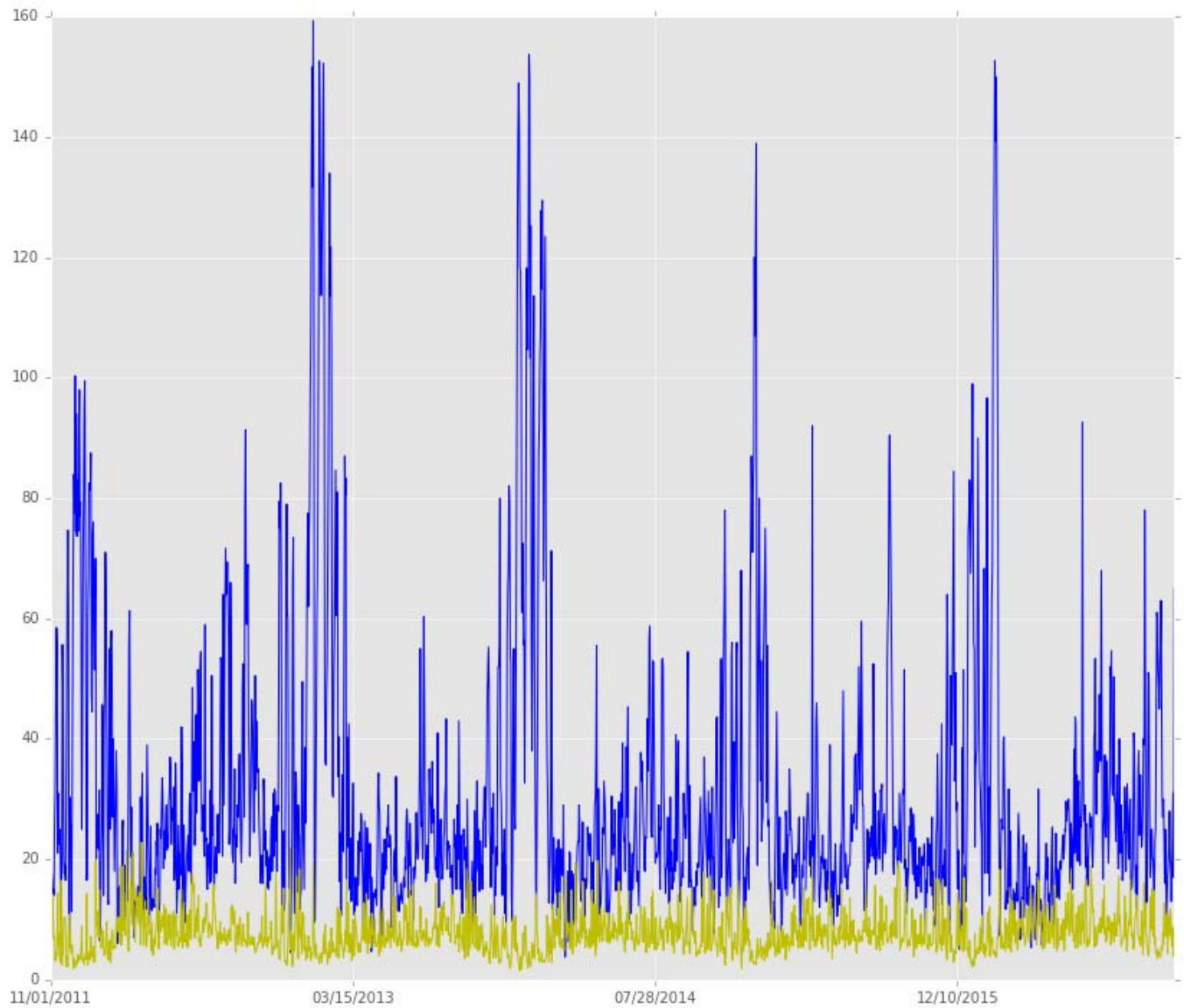
# plot of AQI and Temperature
all_historic_data['DAILY_AQI_VALUE'].plot(grid = True, color = 'b');
all_historic_data['Precipitation (mm)'].plot(grid = True, color = 'r');
all_historic_data.head()
```

Out[8]:

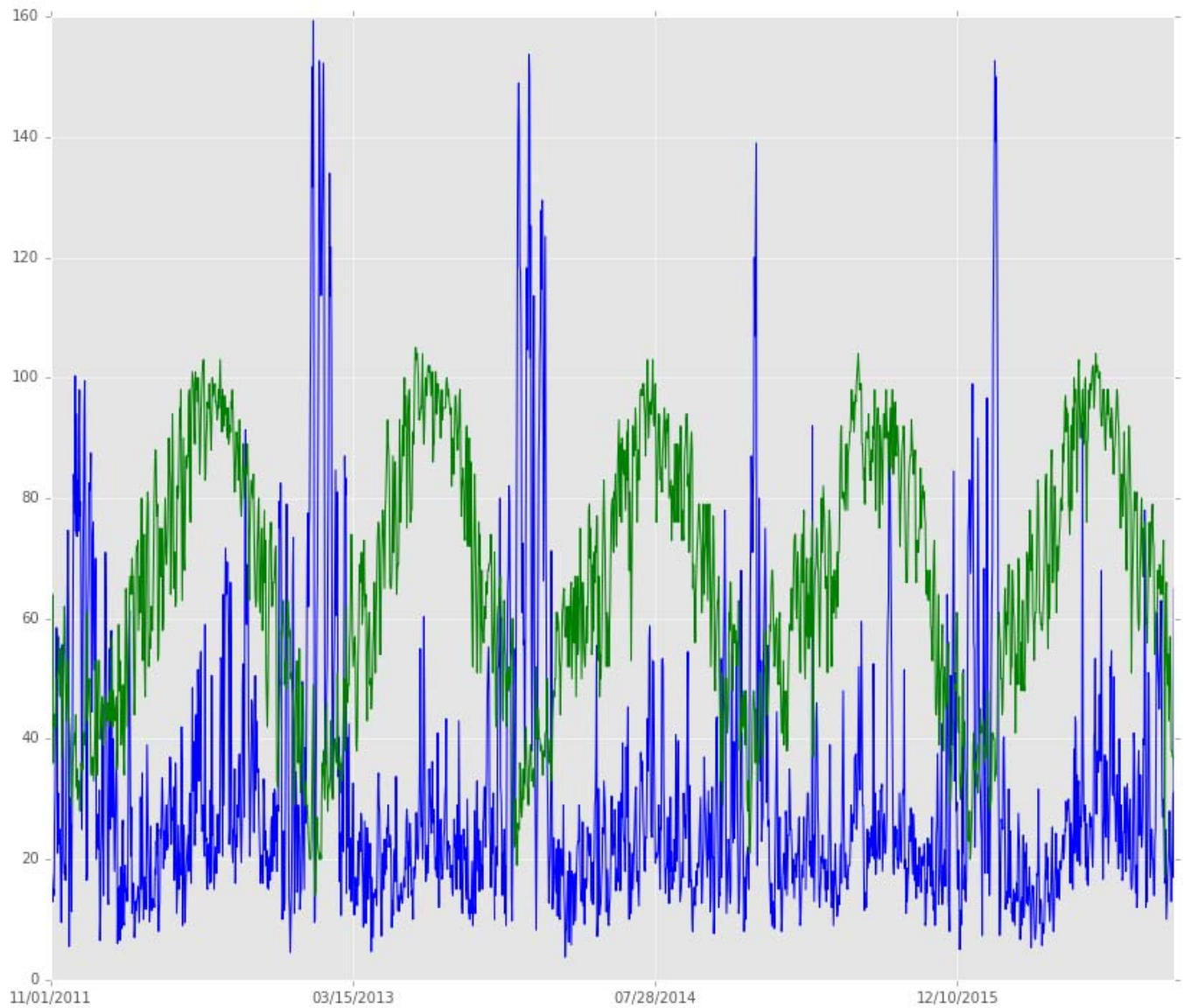
	MAXtemp(F)	MINtemp(F)	AVGtemp(F)	AvgWindSpeed(mph)	MaxWindSpeed(mph)	Preci (mm)
11/01/2011	50	32	41	12.7	35	7.874
11/02/2011	48	27	38	4.7	13	0.000
11/03/2011	58	28	43	12.3	29	0.000
11/04/2011	64	33	49	14.1	31	6.858
11/05/2011	36	29	33	8.1	18	6.096



```
In [9]: # plot of AQI and Avg. Wind speed  
all_historic_data['DAILY_AQI_VALUE'].plot(grid = True, color = 'b');  
all_historic_data['AvgWindSpeed(mph)'].plot(grid = True, color = 'y');
```



```
In [10]: # plot of AQI and Max Temperature
all_historic_data['DAILY_AQI_VALUE'].plot(grid = True, color = 'b');
all_historic_data['MAXtemp(F)'].plot(grid = True, color = 'g');
```

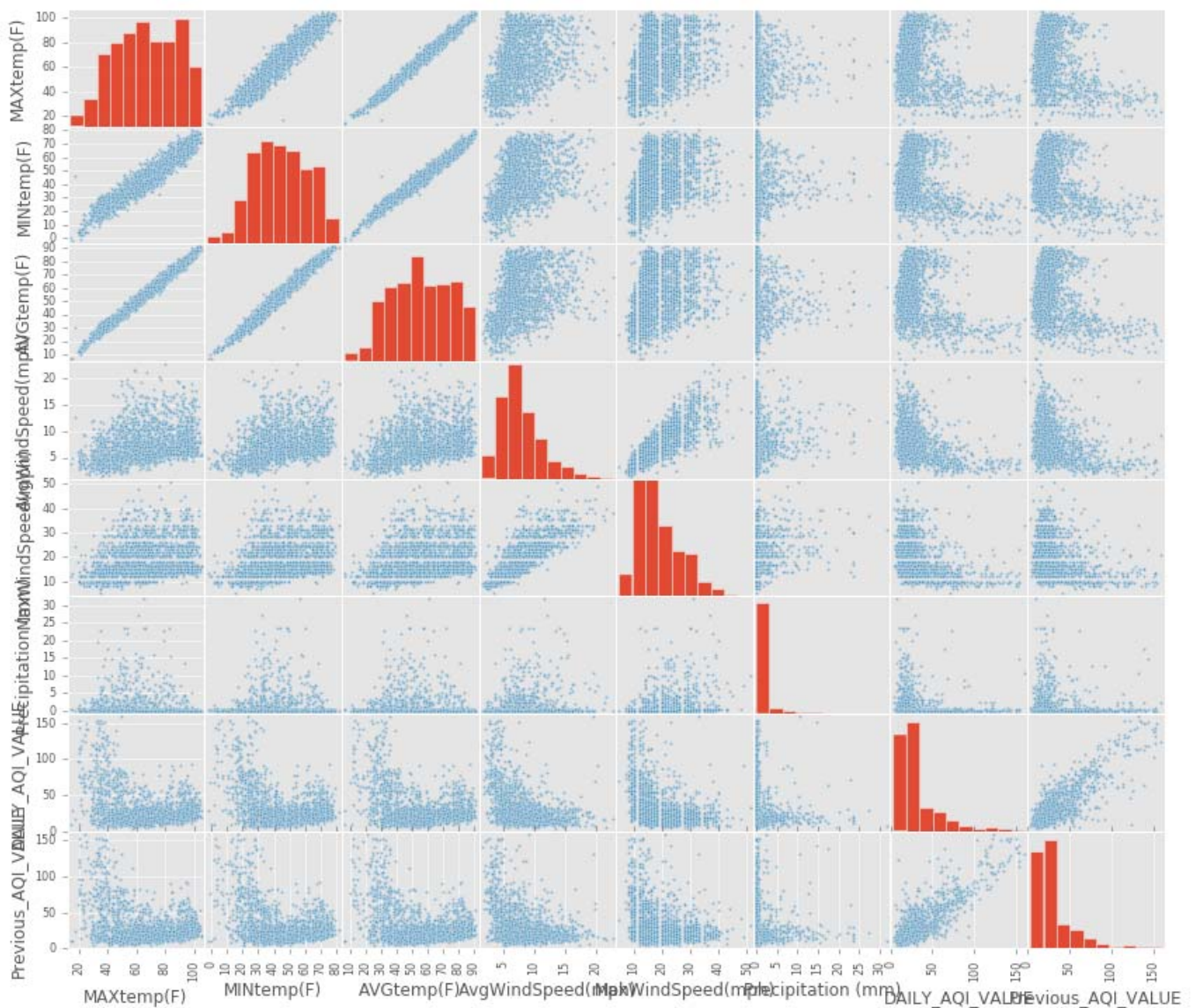


```
In [11]: #visualizing historic data.  
all_historic_data2 = all_historic_data.copy()  
del all_historic_data2["New Date"]  
# print correlations and scatter matrix  
print(all_historic_data2.corr())  
pd.scatter_matrix(all_historic_data2, figsize=(14, 12), diagonal='hist');
```

	MAXtemp(F)	MINtemp(F)	AVGtemp(F)	AvgWindSpeed(mph)	\
MAXtemp(F)	1.000000	0.952716	0.990375	0.331416	
MINtemp(F)	0.952716	1.000000	0.985409	0.364266	
AVGtemp(F)	0.990375	0.985409	1.000000	0.349834	
AvgWindSpeed(mph)	0.331416	0.364266	0.349834	1.000000	
MaxWindSpeed(mph)	0.285559	0.330935	0.309121	0.811983	
Precipitation (mm)	-0.118040	-0.049683	-0.088293	0.165964	
DAILY_AQI_VALUE	-0.333144	-0.358761	-0.348768	-0.357365	
Previous_AQI_VALUE	-0.326513	-0.340124	-0.336826	-0.305431	

	MaxWindSpeed(mph)	Precipitation (mm)	DAILY_AQI_VALUE	\
MAXtemp(F)	0.285559	-0.118040	-0.333144	
MINtemp(F)	0.330935	-0.049683	-0.358761	
AVGtemp(F)	0.309121	-0.088293	-0.348768	
AvgWindSpeed(mph)	0.811983	0.165964	-0.357365	
MaxWindSpeed(mph)	1.000000	0.273777	-0.333686	
Precipitation (mm)	0.273777	1.000000	-0.080051	
DAILY_AQI_VALUE	-0.333686	-0.080051	1.000000	
Previous_AQI_VALUE	-0.270911	-0.029023	0.845956	

	Previous_AQI_VALUE
MAXtemp(F)	-0.326513
MINtemp(F)	-0.340124
AVGtemp(F)	-0.336826
AvgWindSpeed(mph)	-0.305431
MaxWindSpeed(mph)	-0.270911
Precipitation (mm)	-0.029023
DAILY_AQI_VALUE	0.845956
Previous_AQI_VALUE	1.000000



From the correlations and scatter plots, clearly, Previous Day AQI is the strongest predictor. The weather variables tend to be slightly negatively correlated with AQI.

Analysis

Methodology

For our model, we combined the weather predictions and past month's data to create a forecast model for up to 5 days in advance. The historical data was used to train and test the model. We then used cross validation on several different models to see which will work best. Due to the fact that our weather data comes from different sources, only select few features were usable, therefore doing further dimensionality reduction was not needed. The final AQI forecast classifies AQI days according to predicted AQI value:

- $0 \leq \text{Green} < 51$
- $51 \leq \text{Red}$

Implementation

First a very simple model:

```
In [12]: #Simple model based on month of year
# set up functions for numeric classification of AQI and Month.
def AQI_to_numeric(a):
    if a >= 51:
        return 1
    else:
        return 0
def month_classifier(a):
    if a == "01":
        return 1
    else:
        return 0
# create a boolean column's for AQI greater than or less than 51 and month January or not
all_historic_data["Boolean AQI"]=all_historic_data["DAILY_AQI_VALUE"].apply(AQI_to_numeric)
# forward fill any missing values so that the month classifier can be applied.
all_historic_data = all_historic_data.fillna(method = 'ffill')
all_historic_data["Boolean Month"]=all_historic_data["Month"].apply(month_classifier)
```

```
In [13]: # Create and evaluate a simple model to data.
# The simple model assumes all January days are greater than 51 and all other days are less than 51.
confusion_matrix = metrics.confusion_matrix(all_historic_data["DAILY_AQI_VALUE"].apply(AQI_to_numeric),all_historic_data["Boolean Month"])
print('Accuracy on simple model = ', str((confusion_matrix[0][0]+confusion_matrix[1][1])/confusion_matrix.sum()))
print('Percent of days in the last 5 years with AQI value over 51: ', 100*sum(all_historic_data["Boolean AQI"])/len(all_historic_data["Boolean AQI"]))
```

Accuracy on simple model = 0.863440860215

Percent of days in the last 5 years with AQI value over 51: 15.64516129032258

Interpretation of very simple model:

A very simple model which states that all days in January will have an AQI value over 51 and all other days of the year will be under 51 has an accuracy of over 86%. Additionally, since there are only about 15.6% of days in the last 5 years that had AQI values over 51. Even a model that simply always stated that the AQI is under 51 would be 84% accurate.

Final Model Predictions:

```
In [14]: # convert data to a matrix for analysis
y_regres = all_historic_data["DAILY_AQI_VALUE"].as_matrix()
y = all_historic_data["Boolean AQI"].as_matrix()
X = all_historic_data.drop(["DAILY_AQI_VALUE", "Boolean AQI", "Boolean Month", "New Date", "Date", "Month", "Day", "Year"], axis=1).as_matrix()

# create training and testing subsets using stratified sampling since our positive results are sparse.
XTrain, XTest, yTrain, yTest = train_test_split(X, y_regres, random_state=35, test_size=0.4)
XTrain2, XTest2, yTrain2, yTest2 = train_test_split(X, y, random_state=1, stratify=y, test_size=0.4)
```

```
In [15]: # Bayesian Ridge Regression
BR_model = BayesianRidge(compute_score=True)
# train the model
BR_model.fit(XTrain, yTrain)
# check the train model
y_pred_train = BR_model.predict(XTrain)
model_acc = BR_model.score(XTrain, yTrain)
print(" BR training R squared score: ", model_acc)
# test of the model on supset of historical data
y_pred_test = BR_model.predict(XTest)
model_acc = BR_model.score(XTest, yTest)
print(" BR testing R squared score: ", model_acc)
```

```
BR training R squared score:  0.730177540776
BR testing R squared score:  0.729577181841
```

The R squared value indicates that there is a significant amount of correlation between our data and AQI. This value however is not equivalent to an accuracy score.

```

In [16]: # Predict AQI for next 5 days, one day at the time. Using the previous days prediction to predict the next day.

# Set up forecast data for each day starting with day 1 (today) by giving it the last known value
day1["Previous_AQI_VALUE"] = all_historic_data["Previous_AQI_VALUE"][-1]
day1_final = day1.as_matrix()
# predict day 1
y_pred_day1 = BR_model.predict(day1_final)
# predict day 2
day2["Previous_AQI_VALUE"] = y_pred_day1
day2_final = day2.as_matrix()
y_pred_day2 = BR_model.predict(day2_final)
# predict day 3
day3["Previous_AQI_VALUE"] = y_pred_day2
day3_final = day3.as_matrix()
y_pred_day3 = BR_model.predict(day3_final)
# predict day 4
day4["Previous_AQI_VALUE"] = y_pred_day3
day4_final = day4.as_matrix()
y_pred_day4 = BR_model.predict(day4_final)
# predict day 5
day5["Previous_AQI_VALUE"] = y_pred_day4
day5_final = day5.as_matrix()
y_pred_day5 = BR_model.predict(day5_final)

print("By Baysian Ridge model: ")
print("Next 5 days the AQI will be: ", y_pred_day1,y_pred_day2,y_pred_day3,y_pred_day4,y_pred_day5)

```

By Baysian Ridge model:

Next 5 days the AQI will be: [23.96295084] [25.84624208] [29.84834953] [31.26487417] [35.41462775]

```
In [17]: # Forecast classification of Green or Red days using the values from the regression
.  
  
# create a list of the predicted numbers  
predicted_numbers = [y_pred_day1.item(0), y_pred_day2.item(0), y_pred_day3.item(0), y_  
pred_day4.item(0), y_pred_day5.item(0)]  
predicted_days = []  
date_list = finalForecast.index.values  
print("Our model predicts that the next five AQI days will be: ")  
i = 0  
for number in predicted_numbers:  
    print(date_list[i])  
    if number < 51:  
        predicted_days = "Green"  
    else:  
        predicted_days = "Red"  
    print(predicted_days)  
    i = i+1
```

```
Our model predicts that the next five AQI days will be:  
2016-12-04  
Green  
2016-12-05  
Green  
2016-12-06  
Green  
2016-12-07  
Green  
2016-12-08  
Green
```

Conclusion:

Results:

As of Saturday 12-03-2016: According to our model, the AQI forecast gives Green Air Quality for the next 5 days. Since we first began testing this model on 12-01-16, the model has correctly identified a Green AQI day for December 2nd, but incorrectly predicted December 3rd as a Green AQI day, which ended up having an AQI of 65 and should have been a Red day by our model (Yellow day in the classification EPA uses). However, the AirNow.gov website also incorrectly predicted December 3rd to be a green air quality day. So our models seems to not have done any worse then their model. Additionally, AirNow.gov predicts December 4th to have an AQI between 51 and 101 (a Red day within our classification sceme), but our model predicts the AQI to be less then 51, a Green day.

Additional discussion:

- The model we created is very difficult to interpret, but we chose it because it gave values that made sense and had the best R-squared value of the models we tested. Here is a link to the explanation on the mathematical basis of Bayesian Ridge Regression from Scikit-Learn http://scikit-learn.org/stable/modules/linear_model.html#bayesian-regression (http://scikit-learn.org/stable/modules/linear_model.html#bayesian-regression) and a link to an MIT lecture on this topic. <http://www.mit.edu/~9.520/spring09/Classes/class15-bayes.pdf> (<http://www.mit.edu/~9.520/spring09/Classes/class15-bayes.pdf>)
- Regression models do not provide us with an accuracy but rather an R squared score. The R-squared score is similar to a R-squared value one gets from an ordinary linear regression with the exception that it can give values in the range of negative infinity to 1, with 0 indicating no correlation. This means that the only true way to determine how accurate our model is will be to test it out on a day by day basis over the winter. Unfortunately, all days prior to us turning in our assignment were Green Air Quality days, so our model was not tested on days the AQI was over 51. We suspect that as the AQI value increases our model becomes less accurate at predicting the value. This is due to the fact that very few days tend to have high AQI values therefore our data did not have many occurrences to provide for our model to be trained properly. This might be remedied by better data, which we unfortunately did not have access to.
- It is not clear if the AQI value for each Point of Collection site (POC) given in the historical data is an average, minimum or maximum value for that day. Since we averaged the values across POC's we hope that it gives a reasonable estimate for that days air quality index number.
- Although Classification models we tested had accuracies of over 92% they could not be used to obtain predictions for more than 1 day ahead. We therefore did not pursue a classification model.
- We needed to think a lot about how our variables would and could influence our model. We initially created additional columns with moving averages for the AQI, but decided that to drop them. We realized that these 3 variables are strongly effecting our model and were not independent. This could create a situation were our model may be very precise, but would not be very accurate. Since the data was already highly skewed towards the negative result (AQI less then 51) including variables that limited the variability of our predictions would likely exacerbate the problem.
- We attempted to elevate the problems with sparse positive results (AQI over 51) by doing stratified sampling and limiting the over-represented class by narrowing the training data to days from November through February. Unfortunately these methods did not improve our model. An outlier detection model would be good to try, we however have not attempted such a model.
- Although our model is difficult to interpret, it was fast to train and execute.

Additional Models we attempted but did not use:

None of the classification models would work well, even though on the surface they appear to create a very good fit, since we are using the previous day's AQI to predict the next day. In order to create a 5 day forecast we would need to predict the value for each day independently. On the other hand, trying to only use weather data gave poor model performance. Several regression models were also discarded. We needed a model that gave continuous results for the AQI value and the results needed to be reasonable.

Logistic Regression

```
In [18]: # LogisticRegression obtains probabilities then classifies to binary 0,1.

# set up the dataframe using last know AQI value and projecting it forward for all
# forecast days.
finalForecast["Previous_AQI_VALUE"] = all_historic_data["Previous_AQI_VALUE"][-1]
X_final = finalForecast.as_matrix()

#run model
LR_model = LogisticRegression(C=10,solver='liblinear')
LR_model.fit(XTrain2,yTrain2)
y_pred_train = LR_model.predict(XTrain2)
model_acc = metrics.accuracy_score(y_true=yTrain2, y_pred=y_pred_train)
print(" Logistic Regression training accuracy: ",model_acc)
y_pred_test = LR_model.predict(XTest2)
model_acc2 = metrics.accuracy_score(y_true=yTest2, y_pred=y_pred_test)
print(" Logistic Regression testing accuracy: " ,model_acc2)
# predict AQI for next 5 days
y_pred2 = LR_model.predict(X_final)
print(" By Logistic Regression model: Next 5 days the AQI will be: ", y_pred2)

Logistic Regression training accuracy:  0.928315412186
Logistic Regression testing accuracy:  0.934139784946
By Logistic Regression model: Next 5 days the AQI will be:  [0 0 0 0 0]
```

Although, it appears like Logistic Regression would be a good model. This model uses data to train which are not available in the data we used to make the 5 day forecast.

Preceptron Classification:

```
In [19]: # Perceptron classification model
perceptron = Perceptron(n_iter=5000, warm_start=True) # n_iter and warm_start did n
ot seem to effect the performance
perceptron.fit(XTrain2, yTrain2)

y_pred_train = perceptron.predict(XTrain2)
print('Accuracy on training data = ', metrics.accuracy_score(y_true = yTrain2, y_pr
ed = y_pred_train))
y_pred = perceptron.predict(XTest2)
print('Accuracy on test data = ', metrics.accuracy_score(y_true = yTest2, y_pred =
y_pred))
y_pred2 = perceptron.predict(X_final)
print("By Perceptron model: Next 5 days the AQI will be: ", y_pred2)

Accuracy on training data =  0.917562724014
Accuracy on test data =  0.924731182796
By Perceptron model: Next 5 days the AQI will be:  [0 0 0 0 0]
```

This model suffered from the same problems as all classification models. Classification simply wasn't appropriate for what we intended to do.

Decision Tree Regression:

```
In [20]: # Create model for Decision Tree Regression
decisionTree = tree.DecisionTreeRegressor(max_depth=3, min_samples_split=5)
decisionTree = decisionTree.fit(XTrain, yTrain)
y_pred_train = decisionTree.predict(XTrain)
print('Decision Tree Training R squared score = ', metrics.r2_score(y_true = yTrain
, y_pred = y_pred_train))
y_pred = decisionTree.predict(XTest)
print('Decision Tree Testing R squared score = ', metrics.r2_score(y_true = yTest,
y_pred = y_pred))
# predict AQI for next 5 days
# Set up forecast data for each day starting with day 1 (today) by giving it the la
st known value
day1["Previous_AQI_VALUE"] = all_historic_data["Previous_AQI_VALUE"][-1]
day1_final = day1.as_matrix()
# predict day 1
y_pred_day1 = decisionTree.predict(day1_final)
# predict day 2
day2["Previous_AQI_VALUE"] = y_pred_day1
day2_final = day2.as_matrix()
y_pred_day2 = decisionTree.predict(day2_final)
# predict day 3
day3["Previous_AQI_VALUE"] = y_pred_day2
day3_final = day3.as_matrix()
y_pred_day3 = decisionTree.predict(day3_final)
# predict day 4
day4["Previous_AQI_VALUE"] = y_pred_day3
day4_final = day4.as_matrix()
y_pred_day4 = decisionTree.predict(day4_final)
# predict day 5
day5["Previous_AQI_VALUE"] = y_pred_day4
day5_final = day5.as_matrix()
y_pred_day5 = decisionTree.predict(day5_final)

print("By Decision Tree Regression model: ")
print("Next 5 days the AQI will be: ", y_pred_day1,y_pred_day2,y_pred_day3,y_pred_d
ay4,y_pred_day5)

Decision Tree Training R squared score =  0.737661947305
Decision Tree Testing R squared score =  0.697080426709
By Decision Tree Regression model:
Next 5 days the AQI will be:  [ 18.55181886] [ 18.55181886] [ 18.55181886] [ 18.
55181886] [ 18.55181886]
```

The Decision Tree model isn't necessarily bad. We simply felt like the values would not be continuous and therefore the model might perform poorly.

Kernel Ridge Regression:

```

In [21]: # Kernel Ridge Regression with linear kernel
KRR_model = KernelRidge(kernel='linear')
# train the model
KRR_model.fit(XTrain,yTrain)
# check the train model
y_pred_train = KRR_model.predict(XTrain)
model_acc = metrics.r2_score(y_true=yTrain, y_pred=y_pred_train)
print(" KRR training R squared score: " ,model_acc)
# test of the model on supset of historical data
y_pred_test = KRR_model.predict(XTest)
model_acc2 = metrics.r2_score(y_true=yTest, y_pred=y_pred_test)
print(" KRR testing R squared score: " ,model_acc2)

# predict AQI for next 5 days
# Set up forecast data for each day starting with day 1 (today) by giving it the la
st know value
day1["Previous_AQI_VALUE"] = all_historic_data["Previous_AQI_VALUE"][-1]
day1_final = day1.as_matrix()
# predict day 1
y_pred_day1 = KRR_model.predict(day1_final)
# predict day 2
day2["Previous_AQI_VALUE"] = y_pred_day1
day2_final = day2.as_matrix()
y_pred_day2 = KRR_model.predict(day2_final)
# predict day 3
day3["Previous_AQI_VALUE"] = y_pred_day2
day3_final = day3.as_matrix()
y_pred_day3 = KRR_model.predict(day3_final)
# predict day 4
day4["Previous_AQI_VALUE"] = y_pred_day3
day4_final = day4.as_matrix()
y_pred_day4 = KRR_model.predict(day4_final)
# predict day 5
day5["Previous_AQI_VALUE"] = y_pred_day4
day5_final = day5.as_matrix()
y_pred_day5 = KRR_model.predict(day5_final)

print(" By KRR model: Next 5 days the AQI will be: ", y_pred_day1,y_pred_day2,y_pre
d_day3,y_pred_day4,y_pred_day5)

KRR training R squared score:  0.717938725794
KRR testing R squared score:  0.713227131388
By KRR model: Next 5 days the AQI will be:  [ 107.37871839] [ 167.82648794] [ 2
01.43446107] [ 213.79764219] [ 245.87537025]

```

This model was at one point our favorite. However, once we computed things using the previous days AQI the model blew up. It is very unlikely that we would go from an AQI of 25 the day before to an AQI of 260 5 days later.

Support Vector Mashine Regression:

```

In [22]: # Support Vector Mashine Regression
svm_model = svm.SVR(kernel='rbf', C = 50)
svm_model.fit(XTrain,yTrain)
y_pred_train = svm_model.predict(XTrain)
model_acc = metrics.r2_score(y_true=yTrain, y_pred=y_pred_train, multioutput='unifo
rm_average')
print(" SVM training R squared score: " ,model_acc)
y_pred_test = svm_model.predict(XTest)
model_acc2 = metrics.r2_score(y_true=yTest, y_pred=y_pred_test, multioutput='unifor
m_average')
print(" SVM testing R squared score: " ,model_acc2)

# predict AQI for next 5 days
# Set up forecast data for each day starting with day 1 (today) by giving it the la
st know value
day1["Previous_AQI_VALUE"] = all_historic_data["Previous_AQI_VALUE"][-1]
day1_final = day1.as_matrix()
# predict day 1
y_pred_day1 = svm_model.predict(day1_final)
# predict day 2
day2["Previous_AQI_VALUE"] = y_pred_day1
day2_final = day2.as_matrix()
y_pred_day2 = svm_model.predict(day2_final)
# predict day 3
day3["Previous_AQI_VALUE"] = y_pred_day2
day3_final = day3.as_matrix()
y_pred_day3 = svm_model.predict(day3_final)
# predict day 4
day4["Previous_AQI_VALUE"] = y_pred_day3
day4_final = day4.as_matrix()
y_pred_day4 = svm_model.predict(day4_final)
# predict day 5
day5["Previous_AQI_VALUE"] = y_pred_day4
day5_final = day5.as_matrix()
y_pred_day5 = svm_model.predict(day5_final)
print(" By SVM Regression model: Next 5 days the AQI will be: ", y_pred_day1,y_pred
_day2,y_pred_day3,y_pred_day4,y_pred_day5)

SVM training R squared score:  0.885454173537
SVM testing R squared score:  0.0800983775594
By SVM Regression model: Next 5 days the AQI will be:  [ 30.88520398] [ 30.8852
0398] [ 30.88520398] [ 30.88520398] [ 30.88520398]

```

It appears like this model severely overfits the data. Additionally due to the structure of the model, all values predicted tended to remain constant. We therefore did not pursue this direction any further.

Multi-layer Perceptron Regression:

```

In [23]: # Multi-layer Perceptron regressor.
# Each time this block of code gets run the predicted values change.

neuro_model = neural_network.MLPRegressor(hidden_layer_sizes=200, activation='logistic', solver='sgd', shuffle=False, warm_start=False)
neuro_model.fit(XTrain, yTrain)

y_pred_train = neuro_model.predict(XTrain)
print('R squared score training data = ', metrics.r2_score(y_true = yTrain, y_pred = y_pred_train))
y_pred = neuro_model.predict(XTest)
print('R squared score test data = ', metrics.r2_score(y_true = yTest, y_pred = y_pred))

# predict AQI for next 5 days
# Set up forecast data for each day starting with day 1 (today) by giving it the last known value
day1["Previous_AQI_VALUE"] = all_historic_data["Previous_AQI_VALUE"][-1]
day1_final = day1.as_matrix()
# predict day 1
y_pred_day1 = neuro_model.predict(day1_final)
# predict day 2
day2["Previous_AQI_VALUE"] = y_pred_day1
day2_final = day2.as_matrix()
y_pred_day2 = neuro_model.predict(day2_final)
# predict day 3
day3["Previous_AQI_VALUE"] = y_pred_day2
day3_final = day3.as_matrix()
y_pred_day3 = neuro_model.predict(day3_final)
# predict day 4
day4["Previous_AQI_VALUE"] = y_pred_day3
day4_final = day4.as_matrix()
y_pred_day4 = neuro_model.predict(day4_final)
# predict day 5
day5["Previous_AQI_VALUE"] = y_pred_day4
day5_final = day5.as_matrix()
y_pred_day5 = neuro_model.predict(day5_final)
print("By Perceptron regression model: Next 5 days the AQI will be: ", y_pred_day1, y_pred_day2, y_pred_day3, y_pred_day4, y_pred_day5)

R squared score training data = 0.720475419262
R squared score test data = 0.719317505442
By Perceptron regression model: Next 5 days the AQI will be: [ 26.07718685] [ 26.8827265] [ 34.42805677] [ 39.9471893] [ 56.66206662]

```

This is probably one of the weirdest models we have tried. Each time that portion of code was run the numbers would change, we were not sure what was causing this. However, we probably don't have enough data to train a perceptron model.